Information Fusion 122 (2025) 103189

Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus

Full length article

TMF-Net: Multimodal smart contract vulnerability detection based on multiscale transformer fusion

Tengfei Wang, Xiangfu Zhao *, Jiarui Zhang

School of Computer and Control Engineering, Yantai University, Yantai, 264005, China

ARTICLE INFO

Keywords:
Smart contract
Vulnerability detection
Multimodal learning
Multiscale fusion

ABSTRACT

Smart contracts represent a crucial element of the blockchain ecosystem, offering advantages like automation and decentralization. Nevertheless, the occurrence of frequent potential security breaches in smart contracts has led to considerable economic losses and substantial security risks. Although recent approaches have increasingly adopted multimodal strategies, substantial challenges remain in comprehensively extracting information from smart contracts and integrating data across modalities. This hinders capturing both the intricate semantics of smart contracts and the complex interactions across modalities. In this work, we propose **TMF-Net**, a novel multimodal method, using a **M**uti-scale **T**ransformer **F**usion **Net**work to finely detect smart contract vulnerabilities. Our work aims to enhance accuracy by fusing multimodal data and extracting multiscale semantic features. In contrast to conventional techniques, TMF-Net employs an innovative approach that combines three distinct modal information sources: contract graphs, bytecode text, and code images. These inputs are then subjected to a deep feature extraction process, utilizing GCN, Bi-LSTM, and CNN models, respectively. Furthermore, TMF-Net introduces the multiscale transformer fusion module, which facilitates deep multiscale fusion of multimodal information through the multi-head attention mechanism and multi-level encoder structure. This enables the learning of a more comprehensive and fine-grained semantic representation of the code. Experimental results on the validated dataset demonstrate that TMF-Net achieves notable performance gains in the detection of multiple vulnerability types. To illustrate, in the context of reentrancy exploits, the F1-score exhibits a 4.69% improvement over the state-of-the-art methods, which substantiates the efficacy of the multimodal learning and multiscale fusion strategies.

1. Introduction

The rapid advancement of blockchain technology [1] has created unprecedented opportunities for decentralized applications. As a core technology of this innovation, smart contracts play a pivotal role in establishing a trustworthy, transparent, and efficient digital economic ecosystem. A smart contract [2] refers to a self-executing agreement with predefined terms and conditions encoded in computer code that operates autonomously on a blockchain. Smart contracts have a broad spectrum of applications, with their most significant use cases found in the financial sector, including Decentralized Finance (DeFi), digital asset transactions, payment settlements, lending, and insurance. Smart contracts mitigate the inherent risks of manual processes while simultaneously enhancing the efficiency and transparency of trading activities through automated transaction execution. However, the security of smart contracts [3] remains a critical concern, akin to a sword of Damocles, perpetually threatening the stability and growth of the blockchain ecosystem. In recent years, numerous smart contract vulnerabilities and attacks have led to billions of dollars in economic losses, severely undermining user's confidence in blockchain technology. Attackers can exploit vulnerabilities in smart contracts to steal funds, potentially causing substantial financial losses for both users and developers. For instance, The 2016 hack of the Decentralized Autonomous Organization (DAO), resulting from a Reentrancy Attack on its Ethereum smart contract, enabled attackers to misappropriate millions of dollars in Ether [4]. In 2017, a flaw in the Parity wallet's code, known as the unchecked external call vulnerability, enabled an outside user to alter the smart contract's state in ways not originally intended, resulting in the loss of access to substantial cryptocurrency funds held in the impacted wallets [5]. These events underscore the severe repercussions that weaknesses in blockchain systems can inflict upon individuals and entities, thereby amplifying skepticism toward the reliability of blockchain technology.

The widespread adoption of smart contracts offers numerous benefits but presents significant security challenges. To enhance smart

* Corresponding author.
E-mail addresses: wangtengfei8052@gmail.com (T. Wang), xiangfuzhao@gmail.com (X. Zhao).

https://doi.org/10.1016/j.inffus.2025.103189
Received 19 January 2025; Received in revised form 23 March 2025; Accepted 3 April 2025
Available online 11 April 2025

contract security, researchers and developers are continuously refining security auditing and vulnerability detection techniques to ensure their stable and reliable operation. Although numerous vulnerability detection methods [6] have been developed, they still exhibit notable limitations. A comprehensive analysis of existing traditional implementations reveals that current methods for detecting smart contract vulnerabilities can be broadly classified into three main categories: pattern matching, symbolic execution, and fuzz testing. Pattern-matching methods [7,8], which rely on predefined rules, are straightforward to implement but prone to both false positives and false negatives. Symbolic execution methods [9,10] detect vulnerabilities by exhaustively exploring all possible execution paths. However, they are often computationally inefficient and difficult to apply to complex contracts. Fuzz testing methods [11] evaluate contract behavior using random inputs; however, their efficiency and coverage remain unreliable, often requiring manual intervention.

In recent years, deep learning-based approaches [12,13] have been adopted for smart contract vulnerability detection, driven by the increasing application of deep learning across various domains. Traditional rule-based methods [7] depend on predefined rules, which can lead to false positives or false negatives due to their inherent limitations. In contrast, deep learning techniques [14] improve detection accuracy by autonomously learning patterns from data. The nature of smart contract vulnerabilities and their associated attack techniques is continuously evolving. Deep learning models [15] can adaptively learn emerging vulnerability patterns, eliminating the need for manual updates to the rule base. Concurrently, deep learning methods [16] reduce dependence on expert knowledge and improve detection efficiency by autonomously identifying potential vulnerabilities within large-scale data.

Despite their effectiveness, deep learning-based vulnerability detection methods typically rely on unimodal data, such as source code [13] or bytecode [15], thereby failing to fully exploit the rich semantic information available in other modalities. In contrast, multimodal learning overcomes this limitation by integrating information from multiple modalities, enabling a more holistic understanding of smart contracts. Information from diverse modalities can complement each other, addressing the limitations of unimodal data. In this regard, Duy et al. [17] proposed a multimodal smart contract vulnerability detection method, which integrates source code, opcode sequences, and control flow diagrams to achieve a comprehensive analysis of semantic, execution processes, and structured information. However, the feature fusion of this method is mainly realized through simple concatenation and convolution operations, which lack deep interactions of multimodal features, thus restricting the performance of vulnerability detection. Upadhya et al. [18] proposed a multi-modal vulnerability detection method based on interleaved fusion. Although this approach integrates four modes (source code, bytecode, opcode, and intermediate representations) and uses interleaving-based fusion techniques to combine multimodal data, it fell short when dealing with more complex interactions. Additionally, the method lacks data modalities that capture the logical semantics of smart contracts, resulting in incomplete extraction of smart contract information.

However, through an analysis of existing research on multimodal learning, we found that there are two **challenges** to this approach: (i) **Extracting Comprehensive Information about Smart Contracts**. Smart contracts typically involve various types of data and information sources, such as contract graphs, bytecode, and other relevant representations. The information contained in each modality differs, and they may exhibit varying semantic depth and expressiveness. Therefore, how to effectively extract comprehensive and representative features from these modalities, especially when dealing with complex and abstract smart contracts, is a very challenging problem. (ii) **Capture of complex interactive relationships between multiple modalities**. The relationships between the various modalities are often complex and interdependent. For example, instructions in bytecode and node structures in contract diagrams may be interrelated, and these relationships are not simply linear. A key challenge in multimodal learning is how to capture and integrate the interaction and mutual influence between these modes through a suitable model. This model ensures that the deep connections between them are accurately revealed, enhancing overall recognition and robustness.

Recognizing the challenges of existing vulnerability detection methods based on multimodal learning, we propose TMF-Net, a multimodal, fine-grained smart contract vulnerability detection framework leveraging a multiscale Transformer fusion network. Specifically, we first achieve multi-perspective data representation by transforming smart contract code into three distinct modalities: contract graph (Graph), opcode sequence (Text), and code image (Image). Supported by the Multimodal Embedding (ME) module, features from each modality are automatically extracted, effectively capturing the characteristics of the code from diverse perspectives to meet challenge (i). Within the ME module, Graph Convolutional Network (GCN), Bidirectional Long Short-Term Memory network (Bi-LSTM), and Convolutional Neural Network (CNN) are employed to deeply extract features from the contract graph, opcode, and code image, respectively. GCN is employed to extract structural information and node features from the contract graph, effectively capturing the control flow and data flow relationships within the code. This facilitates the identification of complex call logic and dependencies. Bi-LSTM extracts semantic features from opcode sequences, capturing contextual dependencies between opcodes and providing a deeper understanding of the code's execution logic. Simultaneously, CNN is utilized to extract visual features from the code image, capturing its spatial structure and enabling the model to identify code patterns from an image-based perspective. Subsequently, features extracted from the three modalities undergo multiscale and deep-level fusion within the Multiscale Transformer Fusion (MTF) module to address challenge (ii). This facilitates the learning of complex interactions among different modalities. In the MTF module, features from the three modalities are first mapped into a unified feature space. Subsequently, the Multi-Head Attention Mechanism (MHAM) in the MTF module enables parallel feature fusion across diverse subspaces, enabling the extraction of multiscale semantic representations from multimodal data. Finally, the feature representations generated by the MTF module are classified using a multimodal classification (MC) module.

To summarize, we make the following **key contributions**:

- We investigate scenarios where multimodal learning is applied to smart contract vulnerability detection in conjunction with the Transformer model. Our approach combines information from three distinct modalities — contract graph, bytecode text, and code image — and fuses them through the Transformer model. This process generates multi-scale and fine-grained semantic feature representations that better capture complex interactions between various modes and provide novel insights for smart contract security auditing.
- We propose a novel multimodal network framework tailored for smart contract vulnerability detection. Contract graphs, bytecode text, and code images are innovatively integrated within the multimodal network to extract information from contracts, facilitating the generation of comprehensive feature representations of smart contracts.
- Experiments on a validated smart contract dataset demonstrate that our proposed approach achieves a 4.69% improvement in F1-score for detecting reentrancy exploits, surpassing the leading state-of-the-art tool. These results highlight its superiority over traditional methods and existing deep learning approaches for smart contract vulnerability detection.

The remainder of this paper is structured as follows: Section 2 reviews existing studies concerning the detection of vulnerabilities in smart contracts, Section 3 provides a detailed explanation of the design concepts and specifics of our proposed approach, Section 4 presents experimental results to evaluate the effectiveness of our approach and model, and Section 5 concludes the paper and outlines future directions for our research.

2. Related work

This section outlines the foundational research that supports the proposed methodology. It is structured into four key components: smart contract vulnerabilities, traditional vulnerability detection, deep learning-based vulnerability detection, the application of multimodal learning.

2.1. Smart contract vulnerabilities

The common smart contract vulnerabilities that have been identified include reentrancy vulnerabilities, timestamp vulnerabilities, integer overflow vulnerabilities, and delegatecall vulnerabilities [19]. The reentrancy vulnerability, which resulted in the notorious DAO [4] disaster, is one of the vulnerabilities of concern. When smart contracts communicate with external contracts or accounts, reentrancy vulnerabilities occur due to incorrect handling of invocation sequences and state management. This makes it possible for attackers to maliciously re-invoke contracts and withdraw funds from them while they are being executed, resulting in financial losses for users or smart contracts. In Fig. 1, we depict a simplified DAO attack event with the individual actions of the attacker as follows:

- The Hunter first calls the endow function in the contract Reentrancy to deposit 2 Ether.
- Then extract the Ether using the withdraw function.
- When the contract Reentrancy calls the withdraw function and transfers the Ether to the Hunter via the built-in function call. value, the contract Hunter's Fallback function (i.e."function() payable") will be executed automatically.
- The Hunter recursively calls the withdraw function 199 more times in its Fallback function to take out Ether, and the contract Reentrancy will perform 200 coin transfers accordingly, transferring a total of 400 Ethers to the Hunter.

Ultimately, the attacker used their Fallback function to take 398 ethers that were not theirs. Attackers can launch multiple consecutive attacks to get additional resources or money, and reentrancy vulnerabilities are comparatively obscure and challenging to detect. Finding possible reentrant scenarios by examining the contract's dynamic execution pathways is necessary to find reentrancy vulnerabilities. However, there may be uncertainties in interactions between contracts and external contracts or accounts, such as dependencies upon external data sources or dynamic changes in external contract logic. This complexity increases the difficulty of identifying reentrancy problems.

2.2. Traditional vulnerability detection

Conventional approaches primarily rely on expert knowledge and manual analysis. These techniques can be categorized into three main types: pattern matching, symbolic execution, and fuzz testing. Although these methods have demonstrated some success in vulnerability detection, their limitations prevent them from fully addressing the complex requirements of smart contract security.

- Pattern-matching approaches rely on predefined expert rules to identify vulnerabilities in code. These rules are typically based on established vulnerability patterns and best practices, with code analyzed syntactically or semantically to assess potential risks. Slither [20] is a static analysis tool for smart contracts that transforms them into an intermediate representation, SlithIR, and employs established program analysis methods like data flow analysis and taint tracking to extract and refine critical information. SmartCheck [7] is a flexible program analysis tool designed to identify vulnerabilities by converting Solidity code into an XML parse tree and employing XPath schemas for detection. However, this approach has notable limitations, including limited coverage, reliance on detecting only known vulnerability patterns, and challenges in addressing emerging vulnerabilities.
- Symbolic execution-based approaches involve exploring all possible execution paths to identify potential vulnerabilities. It achieves this by converting smart contract code into symbolic expressions and simulating execution using a symbolic execution engine. Oyente [10] is a vulnerability detection tool that uses symbolic execution, analyzing the bytecode of a smart contract along with the current blockchain state. Due to its reliance on a single symbolic execution, Oyente is limited to identifying vulnerabilities that are triggered by a single transaction. Mythril [21] uses a symbolic execution engine to analyze smart contract bytecode, which is combined with an SMT solver to verify the feasibility of potential paths, and tracks data flow through taint analysis to efficiently detect security vulnerabilities in contracts. However, this approach faces significant limitations, including low execution efficiency, challenges in processing complex contracts, and susceptibility to the path explosion problem, which arises due to the exponential increase in execution paths as code complexity grows.
- Fuzz testing-based approaches assess the behavior of smart contracts by generating numerous random inputs to uncover potential vulnerabilities. ContractFuzzer [11], the pioneering vulnerability detection tool built on fuzz testing, is designed to detect seven types of vulnerabilities. However, it is susceptible to detection misses due to the limited coverage of randomly generated test samples. ConFuzzius [22] is the pioneering hybrid fuzz testing tool for smart contracts. While it leverages dynamic data correlation analysis to effectively generate transaction sequences, these sequences have a higher tendency to result in the exposure of concealed erroneous contract states. However, fuzz testing has notable drawbacks, including difficulties in ensuring efficiency and coverage. Additionally, the effectiveness is heavily influenced by the quality and quantity of input data. Result analysis often requires human intervention to identify and classify vulnerabilities.

In summary, traditional detection methods exhibit weaknesses, rendering them insufficient to fully meet the evolving demands of smart contract security. Consequently, the exploration of innovative vulnerability detection methods has emerged as a critical research direction in the field of smart contract security.

2.3. Machine learning based vulnerability detection

Deep learning represents a transformative approach to smart contract vulnerability detection. With the rapid advancement of deep learning technology, researchers [23,24] have begun leveraging its powerful data analysis and pattern recognition capabilities to improve the effectiveness and accuracy of smart contract vulnerability detection. Currently, most deep learning-based approaches focus on unimodal detection, using either source code or bytecode as input for the model. These methods detect exploits by learning syntactic and

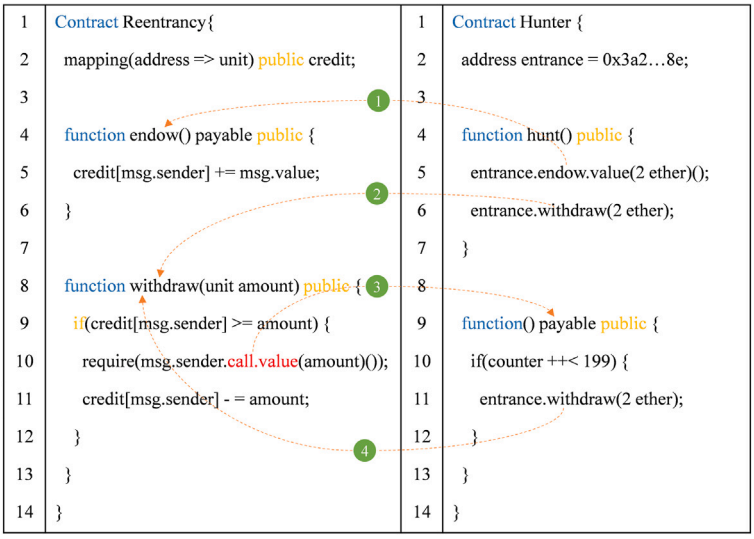

Fig. 1. An example of a basic reentrancy vulnerability.

semantic features of the code. For instance, Qian et al. [13] proposed the pioneering deep learning-based model for smart contract vulnerability detection. It leverages an LSTM network to model Ethernet operand sequences, enabling accurate vulnerability identification. However, extracting features solely from text sequences may overlook structured information and limit contextual relationship modeling, which can affect detection performance. Zhuang et al. [25] developed a graph neural network-based approach to detect vulnerabilities in smart contracts. This approach transforms the source code into a contract graph, which is subsequently analyzed by a graph neural network to identify potential weaknesses. To enhance semantic representation, Zhen et al. [26] employs contract graphs for smart contract vulnerability detection and leverages a dual-attention graph neural network to refine contract feature extraction. Wu et al. [27] developed Peculiar, a novel deep learning-based approach for smart contract vulnerability detection. It leverages critical data flow graphs in conjunction with pre-training techniques to identify vulnerabilities, achieving significant improvements in detection accuracy. However, its reliance on a specific vulnerability type (e.g., reentrancy) may limit its generalizability to other cases, and manual dataset labeling could introduce biases, potentially affecting the model's robustness in broader applications. Zhang et al. [28] proposed the CBGRU model, a hybrid deep learning approach for smart contract vulnerability detection. It combines Word2Vec, FastText, CNN, and Bidirectional Gated Recurrent Unit (BiGRU) to enhance detection accuracy. However, CBGRU may struggle to fully capture the complex semantic and structural information in smart contracts, resulting in an incomplete contextual representation. Moreover, its performance may degrade when detecting vulnerabilities with subtle or indistinct characteristics. However, most existing methods rely on unimodal data, limiting their ability to fully capture the semantic richness embedded in the code. Although multimodal learning has been successfully applied in other domains, limited research has been published on its application in smart contract vulnerability detection.

2.4. Multimodal learning

Multimodal learning leverages data fusion to gain deeper insights across multiple dimensions. Human perception is inherently multimodal, combining information from various senses such as vision, hearing, and touch to construct a comprehensive understanding of the world. Inspired by human perception, multimodal learning seeks to enable machines to process and understand data from multiple modalities. Compared to unimodal approaches, multimodal learning captures a broader spectrum of data features, enhancing model generalization and robustness. Multimodal learning has already achieved notable success in the detection of smart contract vulnerabilities. For instance, Liu et al. [29] proposed the integration of expert patterns with contract graph features for vulnerability detection, leveraging the high precision of expert knowledge as well as the profound insights derived from graph features. However, in the AME network, the combination of expert patterns and graph features is limited to a basic feature merging process, without fully addressing the intricate relationships between expert patterns and graph features. Li et al. [30] introduced FEMD, improving smart contract vulnerability detection by integrating multimodal feature fusion and enhancement techniques, achieving high accuracy through expert-designed graph-based fusion, which captures complex data patterns. Lian et al. [33] proposed a generalizable and efficient multimodal smart contract vulnerability detection framework for large-scale datasets, integrating source code, bytecode, and opcode features using multimodal fusion techniques. This approach achieves high accuracy and scalability while requiring less domain knowledge, achieving significant improvements in performance over traditional methods and unimodal models. Upadhya et al. [31] developed QuadraCode AI, a multimodal Transformer architecture combining source code, bytecode, opcode, and intermediate representations to enhance vulnerability detection in smart contracts. The model achieves high accuracy through advanced fusion techniques like cross-attention and concatenation. Khodadadi et al. [32] explored techniques for identifying vulnerabilities in smart contracts by utilizing different input representations, including deep learning approaches such as BiGRU and word embedding methods. Their findings indicated that the HyMo model outperformed existing models in vulnerability detection. However, the authors restricted their analysis to just two modalities: source code and opcode. Despite the model's superior accuracy compared to previous approaches, it still did not fully meet the anticipated performance levels. Applying multimodal learning to smart contract vulnerability detection enables the comprehensive utilization of multimodal data from smart contracts, facilitating more robust analysis. With the continued advancement of multimodal learning technology, its applications in smart contract security are expected to expand significantly.

Table 1 lists the smart contract vulnerability detection methods and corresponding analysis based on deep learning techniques in recent years.

Table 1
Summary of related work based on deep learning.

Author	Features	Model	Accuracy	Efficiency	Scalability
Qian et al. [13]	Opcode	LSTM	Low	High	Medium
Zhuang et al. [25]	Source Code	TMP, DR-GCN	Medium	High	Medium
Zhen et al. [26]	Opcode	GNN+Attention	Medium	High	Medium
Wu et al. [27]	Source Code	GraphCodeBert	High	High	Medium
Zhang et al. [28]	Source Code	CNN + BiGRU	High	High	Medium
Liu et al. [29]	Source Code + Expert Pattern	TMP + MLP + Attention	High	Medium	Medium
Li et al. [30]	Source Code + Expert Pattern	GAT + MLP + Attention	High	Medium	Medium
Upadhya et al. [31]	Source Code + Bytecode + Opcode + Intermediate representation	BERT + MLP + Attention	High	Medium	High
Khodadadi et al. [32]	Source Code + Opcode	BiGRU	Medium	Medium	High
Lian et al. [33]	Source Code + Bytecode + Opcode	Ensemble Learning	High	Medium	High

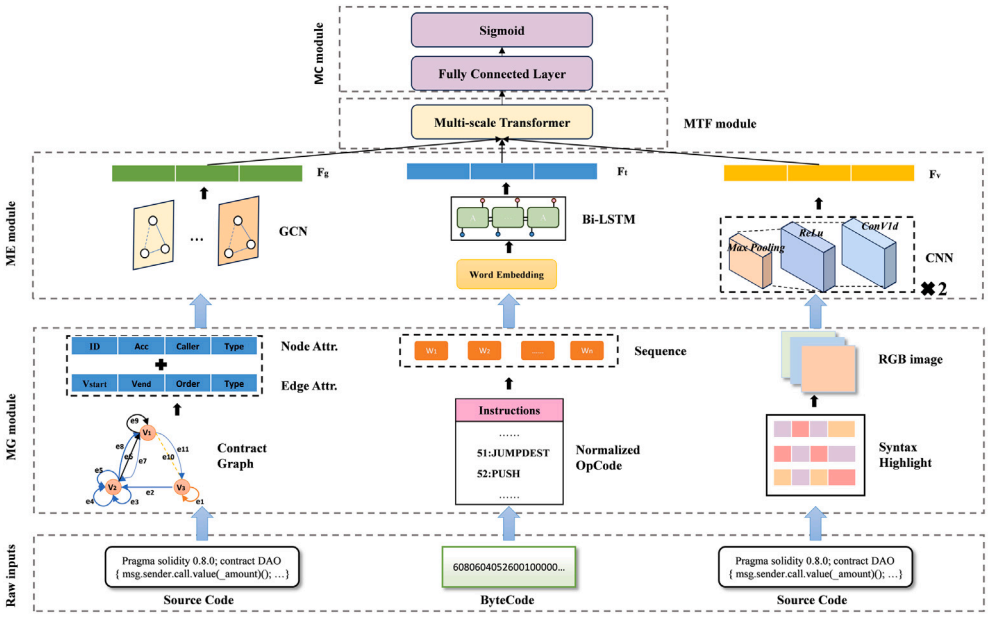

Fig. 2. Architecture of the TMF-Net framework.

3. Methodology

This section introduces and elaborates on the proposed TMF-Net framework. The architecture of the TMF-Net framework is depicted in Fig. 2. The TMF-Net framework comprises four key modules: the Multimodal Generation (MG) module, the Multimodal Embedding (ME) module, the Multiscale Transformer Fusion (MTF) module, and the Multimodal Classification (MC) module. Detailed descriptions of these four modules are provided in the following subsections.

3.1. Overview of TMF-net architecture

The TMF-Net architecture is designed to model the in-depth features of multiple modalities and their complex interrelations, aggregating fine-grained and multiscale features to accurately predict vulnerabilities in smart contracts.

- The MG module converts raw smart contract data into multimodal representations, facilitating a comprehensive understanding of the contract.
- The ME module leverages distinct feature extraction models for each modality to effectively capture deep-level feature representations from the multimodal data.
- The MTF module utilizes a Transformer encoder to map feature vectors from diverse modalities into a unified representation space. These features are fused through the MHAM within the decoder, producing a comprehensive multiscale feature representation.
- The MC module classifies the feature representations generated by the MTF module and outputs the vulnerability detection results.

3.2. Multimodal generation

This module converts raw smart contracts into multimodal data, specifically constructing contract graphs (representing data and control flow), processing bytecode text, and generating code images. The MG module is designed to provide a comprehensive, multidimensional information representation of the smart contract.

3.2.1. Contract graph construction

It is well-known that programs can be represented as symbolic graphs, which encapsulate rich structural and semantic information. Drawing on this concept, smart contracts are translated into graph-structured data for vulnerability detection. According to Liu et al. [25], the source code of a smart contract is represented as a contract graph (CG), capturing data and control dependencies between program statements. Nodes in the graph denote key function calls or variables, while edges represent their execution traces.

To illustrate the modeling of a smart contract as a CG, the construction process is depicted in Fig. 3. Since program elements vary in functional significance, distinct roles are assigned to different program elements (nodes). Additionally, edges are constructed based on the execution order. Accordingly, three types of nodes are extracted: a set of critical nodes, a set of secondary nodes, and a fallback node.

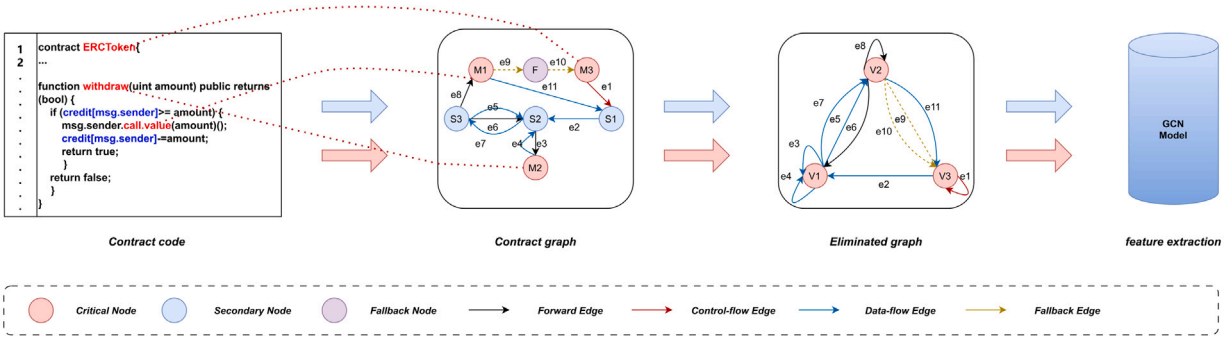

Fig. 3. Process of Contract graph construction.

- A set M of critical nodes, represents the invocations to custom or built-in functions crucial for identifying specific vulnerabilities. To illustrate, in the case of reentrancy vulnerabilities, each critical node $M_i \in M$ models the invocations to the transfer function or the built-in $call.value$ function, which are pivotal for detecting such attacks.
- A set S of secondary nodes, models essential variables, such as user balances and bonus flags, where each node $S_i \in S$ represents a specific variable.
- A fallback node F simulates the fallback functionality of attack contracts, enabling interaction with the contract under test.

Edges are constructed to model relationships between nodes. To capture the rich semantic dependencies between nodes, four types of edges are defined: control flow, data flow, forward and fallback edges. Each edge represents potential paths traversed by the smart contract during testing, with its temporal number indicating the execution order within the contract.

Most graph neural networks are inherently flat in message propagation, often neglecting the central role of certain nodes within the graph. Additionally, the varying graph structures generated from different contract source codes pose challenges for training graph neural networks. To address this, a node elimination process is proposed to normalize the graph structure. Each secondary node $S_i \in S$ is eliminated, and its features are transferred to the nearest critical node. If multiple nearest critical nodes exist for $S_i \in S$, its features are distributed among all such nodes. The fallback node F is processed similarly to secondary nodes. Edges connected to eliminated nodes remain but are adjusted to start or end at the corresponding critical nodes. Critical node features are updated by aggregating features from the adjacent eliminated nodes. We designate the new critical node of M_i as V_i in order to differentiate it from the original critical node and its corresponding critical node following aggregation. Eliminating secondary and fallback nodes emphasizes critical nodes, enhancing the training and learning efficiency of graph neural networks. The transformation of the source code file into a contract graph is represented by Eq. (1).

$$X_G = Construct\_Graph\left(C^S\right) \tag{1}$$

where C^S denotes the original source code and X_G represents the corresponding contract graph.

3.2.2. Bytecode text processing

Bytecode represents the underlying executable code of smart contracts on the blockchain, containing all operational instructions and contract data. On Ethereum, the majority of smart contracts exist in bytecode form. Consequently, extracting semantic information from bytecode is particularly suitable for the runtime environment of smart contracts. To enhance the semantic extraction and improve the model's learning performance, the preprocessing of bytecode involves three primary steps: runtime bytecode acquisition, decompiling bytecode, and normalization. The overall process is illustrated in Fig. 4.

Runtime bytecode acquisition. Smart contract bytecode comprises three components: initialization code (constructor), runtime code, and auxdata, which contains metadata or auxiliary information. Runtime bytecode, which encapsulates the contract's function definitions and implementation logic, is responsible for executing operations and calculations on the blockchain. Isolating the runtime portion is essential because it represents the executable part of the smart contract, which directly interacts with the blockchain and is closely linked to the occurrence of vulnerabilities. Tools like the Solidity compiler (e.g., solc) facilitate this process by allowing users to configure compilation options to generate only the runtime segment of the bytecode. After compilation, the bytecode is loaded into memory, and the runtime section is programmatically extracted and parsed for further analysis. The transformation of a source code file into a bytecode file is represented in Eq. (2).

$$C^B = Compile\left(C^S\right) \tag{2}$$

where C^S denotes the original source code and C^B represents the corresponding bytecode file.

Decompilation. While both the deployment and execution of smart contracts depend on bytecode, its hexadecimal representation is challenging for humans to interpret. To understand the logic of the contract, the bytecode was decompiled into a more readable opcode. Several tools can automatically perform the process of decompiling bytecode. In this work, we employ the EVM disassembler tool, which converts bytecode into opcode sequences. An example of the disassembled opcode is shown in Fig. 4. The transformation of bytecode into its corresponding opcodes is illustrated in Eq. (3).

$$C^O = Disassemble\left(C^B\right) \tag{3}$$

where C^B represents the bytecode file and C^O represents the corresponding opcode file.

Normalization. Instruction normalization mitigates inconsistencies introduced by different compilers or virtual machines, minimizes noise, and enhances model generalization. Jump instructions, represented as $JUMP$, $JUMPI$, $JUMPDEST$, etc., are normalized into a unified format, i.e., $JUMP$. Similarly, stack-pushing instructions, including $PUSH1$, $PUSH2$, and $PUSH32$, etc., which perform the same function but differ in operand size, are unified into a single instruction, i.e., $PUSH$. This normalization simplifies the bytecode, making it more suitable for subsequent analysis and processing. The process of tokenizing the normalized opcode is illustrated in Eq. (4).

$$X_T = Tokenization\left(Norm\left(C^O\right)\right) \tag{4}$$

where X_T denotes the opcode after tokenization and C^O represents the opcode file.

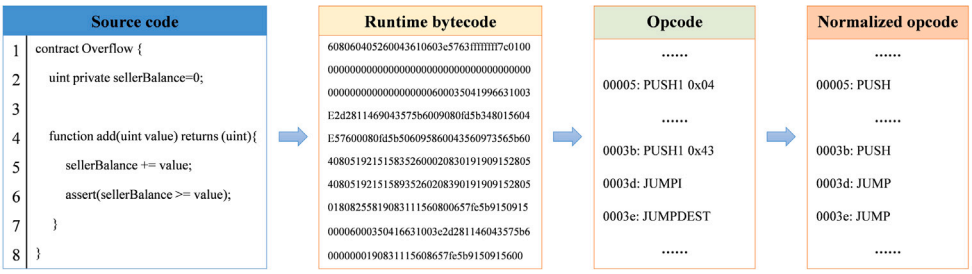

Fig. 4. The process of Bytecode treatment.

3.2.3. Code image generation

Representing smart contracts as images captures the spatial layout and patterns of the code, enabling an intuitive understanding of its structure and potential vulnerabilities. When analyzing smart contract vulnerabilities, both the use of specific keywords and the function call patterns significantly influence the likelihood of vulnerabilities. To effectively identify potential security threats, it is essential to analyze the structural patterns of the code comprehensively. Imaging techniques resolve the spatial layout and patterns of the code, thereby capturing its visual information. This approach not only provides a structured perspective of the contract but also reveals inherent patterns and anomalies. Huang et al. [34] proposed a method that maps hexadecimal bytecode to RGB color codes based on specific rules to generate color images. Building on this, we employed a more innovative strategy to transform contracts into RGB matrices by annotating keywords and other elements of the contract with distinct RGB color markers. These matrices are then saved as images, serving as input for the CNN. Specifically, inspired by the color highlighting of Solidity code in VSCode, Solidity contract elements are categorized into brackets, keywords, operators, identifiers, currency units, comments, and other code elements. A corresponding color image of the code is subsequently generated based on the color markers assigned to each element. The process of converting a source file into a code image is shown in Eq. (5).

$$X_V = Image\_Mapping\left(C^S\right) \tag{5}$$

where C^S represents source code and X_V represents generated code image.

3.3. Multimodal embedding

In the proposed method, multimodal embedding extraction is a critical step. Deep-level information from each modality is captured through embedding extraction processes tailored to each data type. This section describes the procedures for graph embedding extraction, bytecode embedding extraction, and visual embedding extraction in detail.

3.3.1. Graph embedding extraction

The Contract Graph (CG) is embedded using a GCN. GCN effectively captures the complex relationships between nodes and edges in the graph structure, enabling the learning of a comprehensive feature representation for the entire graph. The nodes and edges in the CG are transformed into feature vectors, where the node features encapsulate key functions and core variables, and the edge features reflect data dependencies. GCN propagates and aggregates node features through edges using a message propagation mechanism. Specifically, let the node feature matrix be H and the adjacency matrix be A. The graph convolutional layer is calculated in Eq. (6).

$$H^{(l+1)} = \sigma\left(\sum_{r=1}^{R} AH^{(l)}W_r\right) \tag{6}$$

where $H^{(l)}$ represents the node features at the lth layer, W_r is the weight matrix corresponding to the rth type of relationship, and σ is the activation function. Through this message propagation mechanism, GCN iteratively updates the node features layer by layer to model relationships between nodes. By stacking multiple GCN layers, both local and global features in the graph are aggregated, enabling each node to represent its own information and contextual relationships. The features of the entire graph are subsequently embedded into a fixed-dimensional vector representation through a pooling operation $MaxPool$ in Eq. (7).

$$G_E = MaxPool(\{n_i\}_{i=1}^{|N|}) \tag{7}$$

where n_i denotes the feature vector of node i and N is the total number of nodes in the graph. G_E is the embedding of the entire graph, and $MaxPool$ represents the global maximum pooling function.

3.3.2. Bytecode embedding extraction

Bytecode serves as the foundational code executed by smart contracts on the blockchain, encompassing detailed control logic and sequence information vital for vulnerability detection. To effectively extract rich features from bytecode, a Bi-LSTM network is employed. With its unique gating mechanism, Bi-LSTM can efficiently learn long-term dependencies in sequential data, addressing the limitations of traditional recurrent neural networks when processing long sequences.

Preprocessed bytecode sequences are first converted into word embedding vectors, where each bytecode instruction is mapped into a fixed-dimensional vector. This transformation converts discrete instructions into continuous representations, enhancing the model's ability to capture semantic relationships. In this embedding space, similar instructions are positioned closer to each other, enabling efficient modeling of associations between instructions. Let the bytecode sequence be $X = \{x_1, x_2, \ldots, x_T\}$ on time steps 1, 2, ..., T, where x_t represents the tth word. The word embedding layer maps each instruction $x_t \in X$ into an embedding e_t, as shown in Eq. (8).

$$e_t = Embedding(x_t) \tag{8}$$

where $Embedding$ represents the word embedding algorithm. The embedding vectors $\{e_t\}_{t=1}^{T}$ are passed through a Bi-LSTM layer to capture contextual and sequential relationships within the bytecode. The Bi-LSTM network utilizes three gating mechanisms (input, forget, and output gates) and a memory cell to control the flow and storage of information. Specifically, the input gate determines which new information is added to the memory cell, the forget gate selects outdated information for removal, and the output gate governs the information passed to the next time step. The network computes the hidden state h_i of each unit based on the current input e_t and the hidden state h_{t-1} of the previous unit, as defined in Eqs. (9).

$$\begin{aligned}
i_t &= \sigma(W_i e_t + U_i h_{t-1} + b_i) \\
f_t &= \sigma(W_f e_t + U_f h_{t-1} + b_f) \\
o_t &= \sigma(W_o e_t + U_o h_{t-1} + b_o) \\
g_t &= \tanh(W_c e_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned} \tag{9}$$

where σ denotes the sigmoid activation function, $\odot$ represents the element-wise level multiplication, and tanh is the hyperbolic tangent function. W_i, W_f, W_o, W_c and U_i, U_f, U_o, U_c are weight matrices, while b_i, b_f, b_o, b_c are bias vectors. The Bi-LSTM network outputs a hidden state vector h_t at each time step, encapsulating the sequential information up to that step. As presented in Eq. (10), the hidden state h_T at the final time step T is extracted to serve as the comprehensive embedding of the bytecode sequence.

$$T_E = h_T \tag{10}$$

where T_E represents the embedding of the entire bytecode sequence, encapsulating its contextual and sequential information.

3.3.3. Visual embedding extraction

Smart contract code images, as a visual representation, capture both spatial layout and semantic information. To efficiently extract these features, a CNN is employed. Benefiting from its powerful image feature extraction capability, CNN can capture local patterns and spatial relationships in images to identify structural patterns in source code. To ensure consistency and compatibility with the model's input requirements, images are first preprocessed, including standardization and resizing to a predefined fixed size. This guarantees uniformity across inputs, allowing seamless processing by the CNN. The preprocessed images are passed through the CNN, where multiple convolutional and pooling layers progressively extract features. The convolutional layer extract local patterns by convolving the input image. The activation function $ReLU$ in Eq. (11) is applied to introduce non-linearity.

$$Z = ReLU(Conv(X)) \tag{11}$$

where X is the input image, $Conv$ represents the convolution operation and Z is the feature map after applying $ReLU$. The pooling layer reduce the spatial dimensions of the feature map while retaining the most important information, using a max pooling operation in Eq. (12).

$$C = MaxPool(Z) \tag{12}$$

where $MaxPool$ denotes the max pooling operation and C is the pooled feature map. These layers are stacked to progressively refine and compress visual features, capturing both fine-grained and global spatial characteristics. Once the convolutions and pooling operations are completed, the resulting feature map C is flattened into a one-dimensional vector. As presented in Eq. (13), a fully connected layer is applied to generate the final visual embedding.

$$V_E = ReLU(W_f \cdot Flat(C) + b_f) \tag{13}$$

where $Flat$ denotes the flattening operation. W_f and b_f represent the weights and biases of the fully connected layer, respectively. V_E corresponds to the embedding of the visual modality. The visual embeddings V_E, generated through this process, effectively encodes the structural and spatial patterns in the contract. By progressively extracting and compressing features through convolutional and pooling layers, and refining them in the fully connected layer, the CNN produces a high-dimensional feature vector. This vector encapsulates the visual characteristics of the smart contract.

3.4. Multiscale transformer fusion

The ME module individually extracts embedding representations for each of the three modalities. Subsequently, these embeddings are fused to generate a more comprehensive and nuanced semantic representation of the smart contract. To achieve this, we design the Multiscale Transformer Fusion (MTF) module. The Transformer serves as a bridge between modalities, leveraging its robust sequence modeling capabilities and multi-head attention mechanism to fuse feature representations across modalities at multiple scales, effectively capturing their complex interrelationships. The overall architecture of the MTF module is illustrated in Fig. 5.

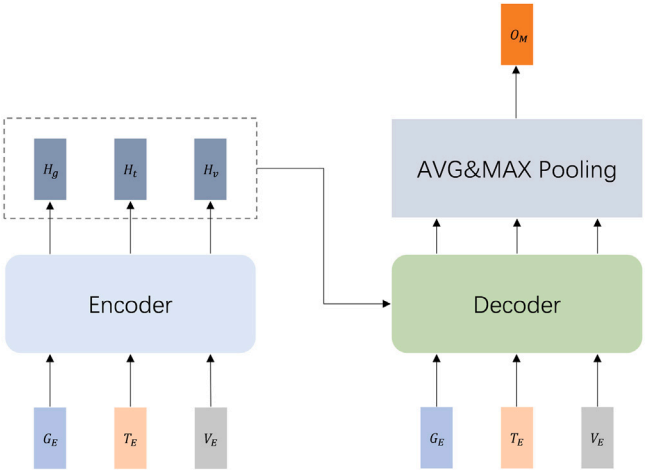

Fig. 5. Overall structure of MTF module.

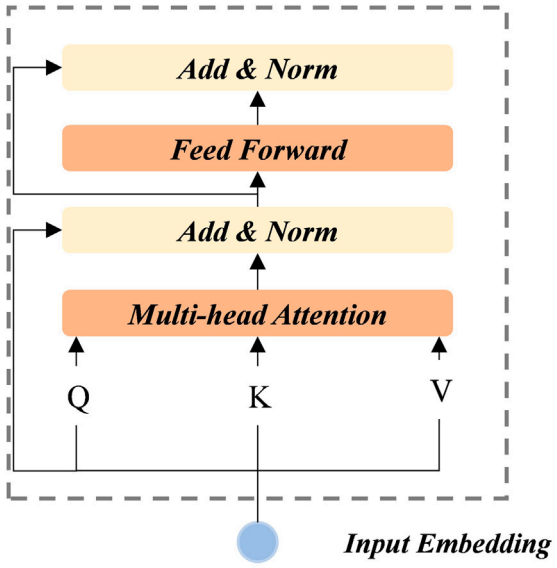

Fig. 6. Basic structure of Transformer.

As illustrated in Fig. 5, the MTF module initially maps feature vectors from diverse modalities into a unified feature space with dimension d through its Encoder, effectively capturing their inter-modal correlations and complementarities. Specifically, the embedding vectors G_E, T_E, and V_E generated by the ME module are fed into the Encoder. After processing by the Encoder, the modality-specific representations are transformed into a unified feature space, denoted as H_g, H_t, and H_v, representing the output of the Encoder. The process described above can be illustrated using Eq. (14).

$$H = (H_g, H_t, H_v) = Encoder(G_E, T_E, V_E) \tag{14}$$

where $H \in \mathbb{R}^{3\times d}$ denotes the encoded output, which is a tuple consisting of three components. H_g, H_t, H_v represent the encoded features of diverse modalities. $Encoder$ represents the encoding algorithm.

The Transformer is a deep learning model that leverages the self-attention mechanism. As illustrated in Fig. 6, the Transformer computes attention through a MHAM. For each attention head i, given a feature X, the feature is initially processed to generate queries (Q_i), keys (K_i), and values (V_i), respectively. Subsequently, attention weights are computed, and the output $head_i$ for each head is derived. The multi-head self-attention mechanism processes distinct subspace information

in parallel across multiple attention heads, subsequently concatenating their outputs to produce the final representation, as described in Eqs. (15).

$$Q_i = W_i^Q X, \qquad K_i = W_i^K X, \qquad V_i = W_i^V X$$
$$Attention(Q_i, K_i, V_i) = softmax\left(\frac{Q_i K_i^T}{\sqrt{d}}\right) V_i$$
$$\text{Multi-Head}(Q, K, V) = Concat(head_1, head_2, \ldots, head_h) W \tag{15}$$

where W_i^Q, W_i^K and W_i^V are learnable projection matrices. $\sqrt{d}$ is the scaling factor to stabilize the gradient. $head_i = Attention(Q_i, K_i, V_i)$. h is the number of self-attention heads. W is a learnable matrix for linear transformations.

The Decoder in the MTF module in Fig. 5 is specifically designed to capture the interrelationships among modalities. By leveraging the multi-head attention mechanism, the relationships among modalities are analyzed from multiple perspectives, facilitating multiscale feature fusion. Specifically, the Decoder receives as input both the features extracted by the Encoder and the embedding representations of each modality produced by the ME module to explore inter-modality correlations. To illustrate, Eqs. (16) presents the formula for calculating the interrelationships between graph modalities and other modalities.

$$Attn(G, H) = softmax\left(\frac{Q_g K_h^T}{\sqrt{d_{K_h}}}\right) V_h$$
$$D_{GH} = L\left(G + \sigma\left(W^T\left(L(G + Attn(G, H))\right)\right) + b\right) \tag{16}$$

where Q_g represents the query from the graph modal G. Similarly, K_h and V_h denote the key and value of feature H from the Encoder output, respectively. L denotes the function LayerNorm, and σ denotes the GELU activation. $Attn$ refers to the self-attention function. D_{GH} is the output of the Decoder. W^T denotes the learnable weight matrix, and b represents the biases.

The Decoder of the MTF module consists of a multi-layer structure built upon the fundamental Transformer architecture. Each layer in the Decoder produces increasingly fine-grained feature representations, effectively capturing deep semantic information. The decoded feature sequence incorporates information from diverse modalities. To enhance global feature extraction, the model applies a pooling operation to the fused feature sequence. Average pooling captures global patterns, whereas maximum pooling highlights and preserves salient features. As outlined in Eq. (17), the output of the pooling operation forms the final multimodal fusion embedding, denoted as O_M.

$$O_M = Pool(D_{GH}, D_{TH}, D_{VH}) \tag{17}$$

where $Pool$ represents the AVG-MAX pooling layer, while D_{TH} and D_{VH} represent the relationships between bytecode text/code image and other modalities, respectively.

3.5. Multimodal classification

By processing the aforementioned modules, a multiscale semantic feature representation integrating three modalities is obtained. This feature is subsequently fed into the Multimodal Classification (MC) module to evaluate and classify the vulnerabilities of the contract. To map the multiscale feature representation to the vulnerability classification space, a fully connected layer is utilized to project the multiscale feature vector O_M into a new feature space, as described in Eq. (18).

$$U = W_o O_M + b_o \tag{18}$$

$$P = softmax(U) \tag{19}$$

where W_o and b_o represent the weight matrix and bias vector of the fully connected layer, respectively. The $softmax$ function in Eq. (19) is applied to transform the output vector U into a probability distribution P, which is then used to determine the vulnerability type of the contract. A general and comprehensive algorithm for TMF-Net's vulnerability detection is presented in Algorithm 1.

Algorithm 1: TMF-Net Vulnerability Detection Algorithm

Input: Smart contract source code C^S
Output: Detection result P

Compile the source code C^S into bytecode C^B, and then disassemble the bytecode C^B to obtain the opcode sequence C^O:
  $C^B = Compile\left(C^S\right)$
  $C^O = Disassemble\left(C^B\right)$
Preprocess to obtain modal representations. Process C^S and C^O to generate the three modalities required by TMF-Net:
  $X_G = Construct\_Graph\left(C^S\right)$
  $X_T = Tokenization\left(C^O\right)$
  $X_V = Image\_Mapping\left(C^S\right)$
Extract deep feature embeddings from each modality using specialized neural networks:
  $G_E = GCN\left(X_G\right)$
  $T_E = BiLSTM\left(Embedding\left(X_T\right)\right)$
  $V_E = CNN\left(X_V\right)$
Project the extracted features into a unified feature space with dimension d:
  $H = \left(H_g, H_t, H_v\right) = Encoder\left(G_E, T_E, V_E\right) \in \mathbb{R}^{3\times d}$
Multiscale Transformer fusion (Decoder):
Process each modality through N Transformer decoder layers ($N \in \{5, 10\}$) to perform multiscale feature fusion:
// M represents the embedding of each modality
  for *each modality* M **do**
    for $n = 1$ *to* N **do**
      $D_{MH} = L\left(M + \sigma\left(W^T\left(L(M + Attn(M, H))\right)\right) + b\right)$
    end
  end
Aggregate the fused multiscale feature representation sequence $D \in \mathbb{R}^{3\times d}$ into a single feature vector:
  $O_M = Pool\left(D_{GH}, D_{TH}, D_{VH}\right)$
Compute the vulnerability prediction:
  $U = W_o O_M + b_o$
  $P = softmax(U)$ //Final result
Return: P

4. Experiments

This section presents the experimental results and performance comparisons of the proposed TMF-Net in the context of smart contract vulnerability detection. The experiments focus on the following three research questions.

- **RQ1**: To what extent can the proposed method effectively detect four types of smart contract vulnerabilities? How does its performance compare to state-of-the-art methods?
- **RQ2**: Is the multimodal learning strategy beneficial for improving the performance of vulnerability detection?
- **RQ3**: How do the multimodal embedding module and the multiscale Transformer fusion module we designed contribute to the overall framework?

The following subsections first detail the experimental setup and then answer each research question sequentially.

4.1. Experimental setup

Dataset description. To comprehensively assess the performance of TMF-Net, we employed a widely recognized benchmark dataset for smart contract vulnerability detection, originally developed by Qian et al. [35] and publicly released. The dataset covers a variety of vulnerability types, namely reentrancy, timestamp dependence, integer overflow/underflow, and delegatecall. It is primarily derived from three sources, i.e., the Ethereum platform (over 96%), GitHub repositories, and blog posts analyzing contracts. Specifically, it includes 42,910 smart contracts, each containing both source code and bytecode. In the 42,910 contracts, 680 contracts have the reentrancy (RE) vulnerability (i.e., label = 1), 2242 contracts possess the timestamp dependence (TD) vulnerability, around 1368 contracts have the integer overflow/underflow (IO) vulnerability, and 136 contracts possess the delegatecall (DT) vulnerability. Each contract sample is annotated with its respective vulnerability types, facilitating the training and evaluation of vulnerability detection tools. Stratified random sampling was applied to partition the dataset into training, validation, and testing sets in a 3:1:1 ratio. Each experiment was conducted five times to ensure reliability, with the average results reported as the final outcomes.

Evaluation Metrics. In this study, four evaluation metrics are employed to assess the model performance, namely accuracy (ACC), recall (RE), precision (PRE), and F1-score (F1). The ACC metric refers to the proportion of the total samples for which the model predicts correctly. The RE metric represents the proportion of positive samples that the model correctly predicts as positive. The PRE metric measures the proportion of samples predicted positive by the model that are actually positive. The F1 metric, as the harmonic mean of precision and recall, provides a balanced measure of both metrics. These four metrics are presented in Eqs. (20).

$$\begin{aligned} ACC &= \frac{TP+TN}{TP+TN+FP+FN} \\ RE &= \frac{TP}{TP+FN} \\ PRE &= \frac{TP}{TP+FP} \\ F1 &= 2\times\frac{PRE\times RE}{PRE+RE} \end{aligned} \tag{20}$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively. These metrics comprehensively measure the performance of TMF-Net in vulnerability detection, revealing the strengths and weaknesses of different aspects. These metrics are employed to evaluate and compare the performance of different tools, clearly demonstrating the advantages of TMF-Net for smart contract vulnerability detection.

Parameter Settings. The proposed model was implemented using PyTorch. All experiments were performed on a workstation configured with a 3.3 GHz Intel Core i9 CPU, an NVIDIA GeForce RTX 2080 Ti GPU, and 64 GB of RAM. The model was trained using cross-entropy loss, the Adam optimizer, and a learning rate scheduler. Through forward and backward propagation, and parameter updates, the model is gradually optimized. A grid search was employed to identify the optimal hyperparameters: the learning rate l was tuned across the values 0.0001, 0.0005, 0.001, 0.002, the hidden layer size h was explored across 64, 128, 256, 512, and the batch size β was varied across 16, 32, 64, 128, respectively. Additionally, the number of attention heads $heads$ was selected from 2, 4, 8, and the number of Transformer layers L was chosen from 2, 4, 6. The hyperparameters yielding the best performance on the training set were selected. The reported results are based on the default hyperparameter settings: (1) $l = 0.0001$, (2) $h = 128$, (3) $\beta = 64$, (4) $heads = 4$, and (5) $L = 4$.

4.2. Performance comparison (RQ1)

Comparison with conventional vulnerability detection tools. TMF-Net was first compared against traditional bug detection tools. Several key tools were selected for comparison: (1) Oyente [10], one of the earliest tools for analyzing smart contracts; (2) SmartCheck [7], a static analysis tool utilizing Abstract Syntax Trees; (3) Osiris [36], a symbolic execution-based vulnerability detection tool; (4) Mythril [21], which combines symbolic execution with taint analysis; (5) Slither [20], a rule-based static analysis tool; (6) sFuzz [37], a fuzz testing-based vulnerability detection tool; and (7) ConFuzzius [22], a hybrid fuzz testing-based detection tool. These tools were chosen for their relevance to provide a comprehensive basis for evaluating the effectiveness of the proposed approach. The following observations can be drawn from Table 2.

- Firstly, the accuracy of traditional detection tools was relatively low across the four vulnerability types. For instance, regarding the reentrancy vulnerability, SmartCheck and Slither achieve only 44.32% and 69.38% accuracy, respectively. This limitation arises primarily from their heavy reliance on rule-base coverage. Rule-based approaches inherently struggle to analyze the complex semantics of smart contracts. When predefined rules fail to encompass specific vulnerability patterns, these tools are unable to detect them, limiting their ability to identify complex vulnerability types.
- Similarly, Oyente and Osiris achieve 65.56% and 52.86% accuracy, respectively, likely due to their reliance on bytecode analysis. Bytecode analysis is restricted to low-level instruction information, making it incapable of capturing the high-level semantic details present in source code.
- In the testing, the fuzz testing tools sFuzz and ConFuzzius achieve accuracies of only 60.42% and 78.02%, respectively. This limitation can be attributed to their reliance on random test cases, which frequently fail to adequately cover the complex execution scenarios inherent in smart contracts.

Notably, TMF-Net produced highly encouraging detection results. Specifically, TMF-Net achieved the best performance across all evaluation metrics for the four vulnerability types, with accuracy improvements of 12.73%, 24.53%, 13.41%, and 8.51%, respectively.

Comparison with Deep Learning-Based approaches. The proposed method was further compared with other deep learning-based tools, namely Vanille-RNN [38], Rechecker [13], DR-GCN [25], TMP [25], DA-GNN [26], AME [29], CBGRU [28], and VulnSense [17]. For a feasible comparison, Vanilla-RNN, DA-GNN, and Rechecker employ bytecode sequences, while GCN, TMP, and CBGRU utilize source code. For tools based on multimodal learning, AME utilizes source code and expert knowledge schemas as inputs, while VulnSense incorporates source code, bytecode, and opcodes as input modalities. The performance of these deep learning models is summarized in Table 2. The experimental results demonstrate that Vanilla-RNN and Rechecker, which process only text sequence information, exhibit relatively poor performance. In contrast, graph neural networks, such as DR-GCN, TMP, DA-GNN, and AME, effectively leverage graph structural information, resulting in superior performance. CBGRU is a hybrid model that outperforms unimodal deep learning tools in detection tasks by leveraging the strengths of various deep learning architectures. The performance of Vulsense, leveraging multimodal learning, outperforms all single-modal methods, thereby demonstrating the superiority of multimodal approaches and highlighting the inherent limitations of single-modal techniques in comprehensively extracting information from smart contracts. Experimental results indicate that TMF-Net outperforms VulnSense, likely because VulnSense relies on simple feature concatenation during multimodal feature fusion, which can only capture linear interactions between modalities. In contrast, TMF-Net

Table 2
Comparison with existing approaches (%).

Methods	Reentrancy				Timestamp				Overflow/Underflow				Delegatecall			
	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1
SmartCheck	44.32	59.04	31.10	40.74	49.00	51.12	66.67	57.87	54.78	66.34	46.65	54.78	53.47	55.37	48.23	51.55
Osiris	51.87	71.78	47.13	56.90	54.70	51.12	61.11	55.67	71.45	51.11	67.46	58.16	n/a	n/a	n/a	n/a
sFuzz	60.42	75.86	30.56	43.57	56.62	34.00	39.17	36.40	72.88	28.57	75.71	41.49	61.84	60.70	52.98	56.58
Mythril	63.31	76.31	43.74	55.61	61.30	50.82	56.30	53.42	n/a	n/a	n/a	n/a	70.41	62.45	71.35	66.60
Oyenete	65.57	66.71	42.11	51.63	64.13	67.37	51.04	58.08	74.18	60.00	60.67	60.34	n/a	n/a	n/a	n/a
Slither	69.38	62.19	68.95	65.40	67.28	62.01	48.90	54.68	n/a	n/a	n/a	n/a	73.47	72.58	56.25	63.38
ConFuzzius	78.02	56.16	59.42	57.74	68.70	56.87	57.48	57.17	76.24	31.11	75.68	44.09	83.16	48.39	86.77	62.13
Vanilla-RNN	62.23	48.79	66.20	56.18	65.51	62.56	70.84	66.44	69.12	49.97	61.34	55.07	64.27	64.55	63.72	64.13
Rechecker	72.48	53.74	61.50	57.36	73.82	58.29	69.75	63.51	70.04	60.40	66.79	63.43	70.94	53.84	81.81	64.94
DR-GCN	74.95	58.78	77.09	66.70	74.32	75.84	68.78	72.14	72.45	68.23	70.46	69.33	76.92	71.79	80.00	75.67
TMP	75.23	65.30	73.45	69.14	76.42	73.21	76.57	74.85	75.06	70.37	73.84	72.06	78.63	79.48	78.15	78.81
DA-GNN	78.62	68.45	76.34	72.18	79.48	76.23	78.95	77.57	76.83	69.74	75.42	72.47	79.87	75.63	81.24	78.33
AME	82.28	74.33	80.76	77.41	84.68	86.49	78.12	82.09	82.88	61.94	73.53	67.24	81.19	74.35	86.13	79.81
CBGRU	83.03	86.29	75.59	80.59	85.20	79.74	86.52	82.99	85.98	80.69	83.04	81.85	85.13	84.04	81.41	82.70
VulnSense	85.47	82.63	84.12	83.37	87.45	81.67	85.32	83.46	87.12	79.84	84.56	82.13	87.45	85.67	84.12	84.89
TMF-Net	**90.75**	**91.45**	**84.92**	**88.06**	**93.23**	**88.15**	**86.59**	**87.36**	**89.65**	**81.40**	**85.24**	**83.28**	**91.67**	**89.49**	**83.33**	**86.30**

Note: 'n/a' indicates that the corresponding tool does not support detection of this vulnerability type.

Table 3
Performance comparison with different modalities (%).

Modality	Reentrancy				Timestamp				Overflow/Underflow				Delegatecall			
	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1
Text-model	64.32	49.12	67.50	56.86	69.79	65.37	76.71	70.59	72.26	50.96	63.00	56.34	65.77	48.72	65.10	55.73
Visual-model	70.05	49.85	69.44	58.04	75.00	76.55	68.00	72.02	77.50	59.41	78.33	67.57	70.68	54.10	74.60	62.72
Graph-model	79.95	58.93	81.09	68.26	80.73	73.70	67.91	70.69	80.64	77.78	79.24	78.50	82.74	67.18	76.19	71.40
TMF-Net	**90.75**	**91.45**	**84.92**	**88.06**	**93.23**	**88.15**	**86.59**	**87.36**	**89.65**	**81.40**	**85.24**	**83.28**	**91.67**	**89.49**	**83.33**	**86.30**

captures the nonlinear and complex interactions between modalities through the MTF module. In terms of accuracy, TMF-Net consistently surpasses other methods across all four vulnerability types. Moreover, the enhanced performance of TMF-Net underscores the effectiveness of our approach in comprehensively extracting information from smart contracts and capturing the intricate interactions across modalities.

4.3. Evaluation on multimodal learning (RQ2)

The proposed multimodal neural network integrates contract graphs (Graph-model), bytecode text (Text-model), and code images (Visual-model). To assess the effectiveness of multimodal learning, unimodal vulnerability detection experiments were conducted, with the results summarized in Table 3. Experimental results indicate that each of the three individual modalities yields meaningful performance in vulnerability detection. The contract graph modality demonstrates the highest detection performance, likely attributable to its capacity to capture deep structural and semantic information of contract code. The visual modality ranks second, while the bytecode text modality shows relatively lower performance. This is likely due to its reliance on instruction-level binary code, which captures only low-level semantic representations of contract code. As illustrated in Table 3, TMF-Net outperforms all unimodal models across all evaluation metrics. For instance, TMF-Net achieves an F1 score improvement from 68.26% to 88.06%. These findings strongly indicate that integrating multimodal information enables the model to capture richer contextual details, thereby enhancing its generalization capability and prediction accuracy for vulnerability detection.

4.4. Ablation study (RQ3)

To comprehensively evaluate the contributions of the individual components of TMF-Net, ablation experiments were conducted by removing specific modalities or fusion mechanisms, thereby allowing an assessment of their respective impacts on detection performance. Initially, individual modalities within TMF-Net were removed to evaluate their contributions to the ME module. The experimental results are summarized in Table 4, with corresponding visual representations shown in Fig. 7.

As detailed in Table 4, removing any modality results in a decline in model performance for vulnerability detection, indicating that all three modalities positively contribute to the detection process. Specifically, the removal of the graph modality leads to accuracy reductions of 9.5%, 12.78%, 13.02%, and 11.02%, respectively, for the four vulnerability types, representing the most significant performance drop. This highlights the critical role of the contract graph in capturing deep semantic relationships, such as function call interactions and data flow directions, which are essential for understanding contract behavior and identifying potential vulnerabilities. Moreover, excluding the bytecode text modality results in F1 score reductions of 9.29%, 10.73%, 9.68%, and 9.08%, respectively, across the four vulnerability types. This finding underscores that bytecode text contributes detailed semantic information, enhancing the model's ability to accurately interpret contract logic and identify vulnerability patterns.

Subsequently, the proposed fusion mechanism was excluded to assess the contribution of the MST module to overall performance. Removing the Transformer-based multimodal fusion mechanism and replacing it with a simpler fusion method resulted in F1 score reductions of 8.71%, 6.52%, 10.98%, and 12.09% for the four vulnerability types, respectively. These findings demonstrate that the Transformer-based fusion mechanism more effectively captures semantic associations across modalities, thereby generating richer code representations and enhancing the model's vulnerability detection performance. The above analysis shows that different types of modal information have varying contributions to smart contract vulnerability detection. The fusion of multimodal information effectively mitigates the limitations of individual modalities, thereby enhancing overall model performance. Moreover, the Transformer-based multimodal fusion technique provides additional performance gains for the model. In summary, the ablation study confirms the effectiveness of the ME and MST modules in TMF-Net. By fusing information from three modalities and leveraging the Transformer-based fusion mechanism, TMF-Net demonstrates an enhanced capability to understand smart contract behavior and accurately identify potential vulnerabilities.

Table 4
Results of ablation experiments (%).

Model	Reentrancy				Timestamp				Overflow/Underflow				Delegatecall			
	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1	ACC	RE	PRE	F1
G - w/o	81.25	69.69	76.03	72.72	80.45	82.68	77.68	80.10	76.63	64.86	77.50	70.62	80.65	67.18	80.00	73.03
Trans - w/o	85.16	75.94	83.08	79.35	83.33	82.22	79.51	80.84	80.69	66.67	78.98	72.30	81.99	71.03	77.69	74.21
T - w/o	85.94	86.41	72.37	78.77	85.46	75.26	78.06	76.63	85.15	73.47	73.73	73.60	83.78	81.54	73.33	77.22
V - w/o	87.24	76.15	76.99	76.57	86.42	82.13	81.72	81.92	87.64	79.48	77.17	78.31	88.24	78.97	80.95	79.95
TMF-Net	**90.75**	**91.45**	**84.92**	**88.06**	**93.23**	**88.15**	**86.59**	**87.36**	**89.65**	**81.40**	**85.24**	**83.28**	**91.67**	**89.49**	**83.33**	**86.30**

Notes: G - w/o, T - w/o, and V - w/o denote the elimination of individual modalities, respectively. And Trans - w/o denotes the elimination of the Transformer-based multimodal fusion mechanism.

Table 5
Performance comparison of alternative fusion mechanisms (%).

Variants	Reentrancy			Timestamp			Overflow/Underflow			Delegatecall		
	ACC	F1	ACC-Dec	ACC	F1	ACC-Dec	ACC	F1	ACC-Dec	ACC	F1	ACC-Dec
DVF	71.35	58.66	−19.40	72.92	74.29	−20.31	72.81	60.39	−16.84	72.32	65.01	−19.35
DWV	72.92	60.50	−17.83	74.48	74.77	−18.75	73.75	61.26	−15.90	78.87	66.92	−12.80
FDC	84.90	73.02	−5.85	83.85	78.86	−9.38	83.76	66.81	−5.89	80.95	77.57	−10.72
FWC	86.98	76.00	−3.77	84.90	80.15	−8.33	84.90	69.38	−4.75	86.16	78.71	−5.51
TMF-Net	**90.75**	**88.06**	–	**93.23**	**87.36**	–	**89.65**	**83.28**	–	**91.67**	**86.30**	–

Notes: the ACC-Dec indicates the percentage decrease in accuracy compared to the TMF-Net.

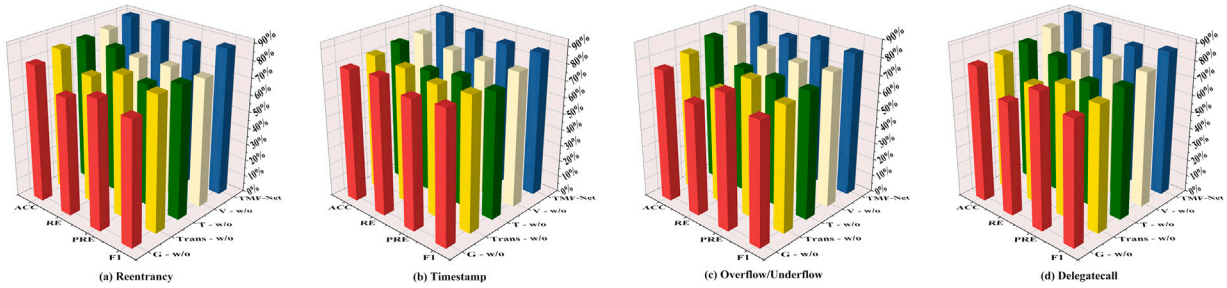

Fig. 7. Visual analysis of ablation experiments.

To further evaluate the effectiveness of the MST module, we compare TMF-Net with four mainstream multimodal fusion techniques, encompassing both feature-level and decision-level fusion.

- Feature-level fusion approaches include: (1) Direct Feature Concatenation (FDC), where the feature vectors of the three modalities are concatenated into a single vector and input into a fully connected layer for classification; and (2) Feature Weighted Concatenation (FWC), where modality-specific weights are assigned to the feature vectors, reflecting the relative importance of each modality.
- Decision-level fusion methods include: (1) Decision Voting Fusion (DVF), which involves training independent classifiers for each modal feature vector, then determining the final prediction result through majority voting; and (2) Decision Weighted Voting (DWV), which assigns weights to the predictions of each classifier based on the modality's contribution to the final decision.

Table 5 shows the performance comparison of TMF-Net and the four fusion approaches in detecting vulnerabilities, where "ACC-Dec" represents the percentage decrease in accuracy relative to TMF-Net.

Results from Table 5 indicate that feature-level fusion methods (FDC and FWC) outperform decision-level fusion methods (DVF and DWV). This outcome is likely because feature-level fusion combines information at earlier stages of model processing, preserving richer feature interactions. In contrast, decision-level fusion combines information at the final decision stage, potentially leading to a loss of critical modal interactions. Furthermore, FWC slightly outperforms FDC, suggesting that weighting feature vectors from different modalities more effectively reflects their relative importance, thereby improving model performance.

Notably, TMF-Net achieves substantial improvements over the other four fusion methods in detecting the four vulnerability types. This can be attributed to the MST module's ability to effectively resolve modal inconsistencies and capture complex interrelations among modalities. Specifically, the MST module first aligns the three modalities in the high-level feature space. It then fuses modality-specific information via different perspectives through the multi-head attention mechanism. Finally, it utilizes a multi-layer decoder to obtain a deep-level semantic representation of the contract. In summary, the MST module employed in TMF-Net effectively fuses multimodal information, significantly enhancing the model's ability to detect vulnerabilities in smart contracts.

5. Conclusion

In this paper, we propose TMF-Net, an innovative framework for smart contract vulnerability detection leveraging multimodal fusion. By effectively integrating textual, visual, and structural information from smart contracts, TMF-Net achieves significant improvements in detection accuracy and robustness over state-of-the-art methods. The primary contribution of this work lies in the development of a tailored network architecture that enhances multimodal feature extraction and fusion capabilities. It leverages advanced deep learning techniques to tackle the complexities of smart contract analysis. TMF-Net can serve as an internal security audit tool for the automatic detection of smart contract vulnerabilities. It reduces the reliance on manual audits while ensuring the stable operation of enterprise blockchain systems through high accuracy and robust multimodal analysis capabilities. For instance, it can enhance transaction transparency and security in supply chain finance applications. One limitation of TMF-Net is its

current focus on Solidity-based smart contracts on the Ethereum platform, with its adaptability to other programming languages still under investigation. Additionally, the model's high computational complexity could limit its performance in real-time detection applications. This study opens up several promising research directions for smart contract vulnerability detection. Future research could focus on adapting TMF-Net to emerging smart contract languages or developing more advanced fusion mechanisms to further enhance detection performance. Moving forward, we plan to explore additional modalities and investigate more sophisticated multimodal fusion techniques to further advance blockchain security.

CRediT authorship contribution statement

Tengfei Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Xiangfu Zhao:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition. **Jiarui Zhang:** Writing – review & editing, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by National Natural Science Foundation of China (Grant No. 61972360), and Shandong Provincial Natural Science Foundation, China (Grant No. ZR2024MF111).

Data availability

Data will be made available on request.

References

[1] F. Casino, T.K. Dasaklis, C. Patsakis, A systematic literature review of blockchain-based applications: Current status, classification and open issues, Telemat. Inform. 36 (2019) 55–81.

[2] Z. Zheng, S. Xie, H.-N. Dai, W. Chen, X. Chen, J. Weng, M. Imran, An overview on smart contracts: Challenges, advances and platforms, Future Gener. Comput. Syst. 105 (2020) 475–491.

[3] Y. Huang, Y. Bian, R. Li, J.L. Zhao, P. Shi, Smart contract security: A software lifecycle perspective, IEEE Access 7 (2019) 150184–150202.

[4] S. Wang, L. Ouyang, Y. Yuan, X. Ni, X. Han, F.-Y. Wang, Blockchain-enabled smart contracts: Architecture, applications, and future trends, IEEE Trans. Syst. Man Cybern.: Syst. 49 (11) (2019) 2266–2277.

[5] E. Mik, Smart contracts: Terminology, technical limitations and real world complexity, Law, Innov. Technol. 9 (2) (2017) 269–300.

[6] G. Sun, C. Jiang, J. Shen, Y. Zhang, Smart contract vulnerability detection methods: A survey, in: International Conference on Blockchain and Trustworthy Systems, Springer, 2024, pp. 179–196.

[7] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, Y. Alexandrov, Smartcheck: Static analysis of ethereum smart contracts, in: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain, 2018, pp. 9–16.

[8] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Buenzli, M. Vechev, Securify: Practical security analysis of smart contracts, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 67–82.

[9] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, A. Dinaburg, Manticore: A user-friendly symbolic execution framework for binaries and smart contracts, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering, IEEE, 2019, pp. 1186–1189.

[10] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, A. Hobor, Making smart contracts smarter, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 254–269.

[11] B. Jiang, Y. Liu, W.K. Chan, ContractFuzzer: Fuzzing smart contracts for vulnerability detection, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, 2018, pp. 259–269.

[12] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: NDSS, 2018.

[13] P. Qian, Z. Liu, Q. He, R. Zimmermann, X. Wang, Towards automated reentrancy detection for smart contracts based on sequential models, IEEE Access 8 (2020) 19685–19695.

[14] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.

[15] C. Sendner, H. Chen, H. Fereidooni, L. Petzi, J. König, J. Stang, A. Dmitrienko, A.-R. Sadeghi, F. Koushanfar, Smarter contracts: Detecting vulnerabilities in smart contracts with deep transfer learning, in: NDSS, 2023.

[16] X. Yu, H. Zhao, B. Hou, Z. Ying, B. Wu, DeeSCVHunter: A deep learning-based framework for smart contract vulnerability detection, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE, 2021, pp. 1–8.

[17] P.T. Duy, N.H. Khoa, N.H. Quyen, L.C. Trinh, V.T. Kien, T.M. Hoang, V.-H. Pham, VulnSense: Efficient vulnerability detection in ethereum smart contracts by multimodal learning with graph neural network and language model, Int. J. Inf. Secur. 24 (1) (2025) 48.

[18] J. Upadhya, A. Sainju, K. Upadhyay, S. Poudel, M.N. Hasan, K. Poudel, J. Ranganathan, VulnFusion: Exploiting multimodal representations for advanced smart contract vulnerability detection, in: 2024 6th International Conference on Blockchain Computing and Applications, BCCA, IEEE, 2024, pp. 505–515.

[19] G. Ramakrishnan, M. Rehan, G. Sujatha, Solidity vulnerability scanner, in: 2022 International Conference on Data Science, Agents & Artificial Intelligence, ICDSAAI, Vol. 1, IEEE, 2022, pp. 1–5.

[20] J. Feist, G. Grieco, A. Groce, Slither: a static analysis framework for smart contracts, in: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, WETSEB, IEEE, 2019, pp. 8–15.

[21] B. Mueller, A framework for bug hunting on the ethereum blockchain, 2017, Available at: https://github.com/ConsenSys/mythril.

[22] C.F. Torres, A.K. Iannillo, A. Gervais, R. State, ConFuzzius: A data dependency-aware hybrid fuzzer for smart contracts, in: 2021 IEEE European Symposium on Security and Privacy, EuroS&P, IEEE, 2021, pp. 103–119.

[23] L. He, X. Zhao, Y. Wang, J. Yang, X. Sun, GraphSA: Smart contract vulnerability detection combining graph neural networks and static analysis, in: ECAI 2023, IOS Press, 2023, pp. 1020–1027.

[24] Y. Wang, X. Zhao, L. He, Z. Zhen, H. Chen, ContractGNN: Ethereum smart contract vulnerability detection based on vulnerability sub-graphs and graph neural networks, IEEE Trans. Netw. Sci. Eng. 11 (6) (2024) 6382–6395.

[25] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, Q. He, Smart contract vulnerability detection using graph neural networks, in: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence, 2021, pp. 3283–3290.

[26] Z. Zhen, X. Zhao, J. Zhang, Y. Wang, H. Chen, DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network, Comput. Netw. 242 (2024) 110238.

[27] H. Wu, Z. Zhang, S. Wang, Y. Lei, B. Lin, Y. Qin, H. Zhang, X. Mao, Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques, in: 2021 IEEE 32nd International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2021, pp. 378–389.

[28] L. Zhang, W. Chen, W. Wang, Z. Jin, C. Zhao, Z. Cai, H. Chen, Cbgru: A detection method of smart contract vulnerability based on a hybrid model, Sensors 22 (9) (2022) 3577.

[29] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, X. Wang, Combining graph neural networks with expert knowledge for smart contract vulnerability detection, IEEE Trans. Knowl. Data Eng. 35 (2) (2021) 1296–1310.

[30] J. Li, R. Li, Y. Lu, B. Cai, Y. Cao, H. Lin, X. Li, C. Li, FEMD: Feature enhancement-aided multimodal feature fusion approach for smart contract vulnerability detection, in: 2024 IEEE 30th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2024, pp. 641–648.

[31] J. Upadhya, K. Upadhyay, A. Sainju, S. Poudel, M.N. Hasan, K. Poudel, J. Ranganathan, QuadraCode AI: Smart contract vulnerability detection with multimodal representation, in: 2024 33rd International Conference on Computer Communications and Networks, ICCCN, IEEE, 2024, pp. 1–9.

[32] M. Khodadadi, J. Tahmoresnezhad, Hymo: Vulnerability detection in smart contracts using a novel multi-modal hybrid model, 2023, arXiv preprint arXiv: 2304.13103.

[33] W. Lian, Z. Bao, X. Zhang, B. Jia, Y. Zhang, A universal and efficient multi-modal smart contract vulnerability detection framework for big data, IEEE Trans. Big Data (2024).

[34] T. Hsien-De Huang, H.-Y. Kao, R2-d2: Color-inspired convolutional neural network (cnn)-based android malware detections, in: 2018 IEEE International Conference on Big Data (Big Data), IEEE, 2018, pp. 2633–2642.

[35] P. Qian, Z. Liu, Y. Yin, Q. He, Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 2220–2229.

[36] C.F. Torres, J. Schütte, R. State, Osiris: Hunting for integer bugs in ethereum smart contracts, in: Proceedings of the 34th Annual Computer Security Applications Conference, 2018, pp. 664–676.

[37] T.D. Nguyen, L.H. Pham, J. Sun, Y. Lin, Q.T. Minh, Sfuzz: An efficient adaptive fuzzer for solidity smart contracts, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020, pp. 778–788.

[38] W.J.-W. Tann, X.J. Han, S.S. Gupta, Y.-S. Ong, Towards safer smart contracts: A sequence learning approach to detecting security threats, 2018, arXiv preprint arXiv:1811.06632.